

Análisis, requisitos, validación, trazabilidad, gestión

Breve descripción:

El componente formativo aborda los procesos relacionados con el análisis, la especificación, la validación y la gestión de requisitos de software. Se estudian técnicas que permiten identificar, documentar, evaluar y controlar los requisitos durante el desarrollo de un sistema, con el fin de asegurar que las soluciones tecnológicas respondan adecuadamente a las necesidades del usuario y del negocio.

Tabla de contenido

Introducción	4
1. Análisis de requisitos de software.....	7
1.1. Técnicas de análisis de requisitos	8
1.2. Priorización de requisitos.....	10
1.3. Matrices de trazabilidad	12
1.4. Descomposición funcional	15
2. Especificación de requisitos de software	18
2.1. Estándares internacionales para la especificación de requisitos	18
2.2. Plantillas ERS (Especificación de Requisitos del Software)	20
2.3. Técnicas de documentación de requisitos en proyectos tradicionales ..	22
2.4. Técnicas de documentación de requisitos en proyectos ágiles.....	24
3. Validación de requisitos de software	26
3.1. Criterios de validación de requisitos	27
3.2. Técnicas de validación de requisitos	29
3.3. Revisiones, auditorías y prototipos	31
3.4. Manual de usuario como mecanismo de validación	33
4. Gestión de requisitos en proyectos de software	36
4.1. Contratos de software: requisitos fijos y variables	38

4.2. Herramientas para la gestión de requisitos	41
4.3. Tipos y características de herramientas de gestión de requisitos	44
4.4. Control y gestión de cambios en los requisitos	47
Síntesis	49
Glosario	50
Referencias bibliográficas	52
Créditos	53

Introducción

El desarrollo de software requiere comprender con precisión las necesidades del usuario y del negocio antes de construir una solución tecnológica. En este contexto, el análisis, la especificación y la validación de requisitos se convierten en actividades fundamentales dentro de los proyectos de ingeniería de software, ya que permiten identificar, documentar y verificar las características que debe cumplir el sistema.

Este componente formativo aborda las técnicas utilizadas para analizar requisitos, incluyendo la priorización, las matrices de trazabilidad y la descomposición funcional, herramientas que permiten organizar y comprender de manera estructurada las funcionalidades del sistema. Asimismo, se presentan los fundamentos para la especificación de requisitos, considerando estándares internacionales y el uso de plantillas de Especificación de Requisitos del Software (ERS), junto con técnicas de documentación aplicadas tanto en metodologías tradicionales como en enfoques ágiles.

Igualmente, se estudian los criterios y técnicas de validación de requisitos, orientados a garantizar que los requisitos definidos sean claros, completos, consistentes y viables antes de su implementación. Finalmente, se analizan aspectos relacionados con la gestión de requisitos en proyectos de software, incluyendo los contratos de software y el uso de herramientas especializadas para su administración.

El estudio de estos elementos permite fortalecer la calidad del proceso de desarrollo, mejorar la comunicación entre los actores del proyecto y reducir riesgos asociados a requisitos incompletos o mal definidos.

Para comprender la importancia del contenido y los temas abordados, se recomienda acceder al siguiente video:

Video 1. Análisis, especificación y validación de requisitos de software



Enlace de reproducción del video

Transcripción del video: Análisis, especificación y validación de requisitos de software

La evolución del desarrollo de software ha demostrado que uno de los factores más determinantes para el éxito de un proyecto es comprender con claridad qué necesita realmente el usuario antes de comenzar a programar. En las primeras décadas de la computación, muchos proyectos fallaban porque los sistemas se desarrollaban sin una definición clara de sus funcionalidades, lo que generaba retrasos, sobrecostos y productos finales que no respondían a las necesidades reales de las organizaciones.

Ante esta problemática surgió la disciplina conocida como ingeniería de requisitos, la cual se enfoca en identificar, analizar, documentar, validar y gestionar los requisitos que debe cumplir un sistema de software. Con el crecimiento de los sistemas informáticos durante las décadas de 1970 y 1980, esta área comenzó a consolidarse como una fase fundamental dentro del proceso de desarrollo de software.

Con el tiempo, diferentes organismos internacionales han establecido estándares que orientan la forma en que se deben documentar y gestionar los requisitos en proyectos tecnológicos, entre ellos los desarrollados por la International Organization for Standardization y el Institute of Electrical and Electronics Engineers, los cuales promueven buenas prácticas para garantizar claridad, trazabilidad y control de cambios en los requisitos del sistema.

En este contexto, el análisis de requisitos permite comprender las necesidades del usuario mediante diversas técnicas; la especificación organiza y documenta dichas necesidades de manera estructurada; la validación verifica que los requisitos definidos correspondan realmente a lo que el sistema debe cumplir; y la gestión de requisitos facilita el control y seguimiento de los cambios que pueden surgir durante el desarrollo.

1. Análisis de requisitos de software

El análisis de requisitos de software corresponde al proceso mediante el cual se examinan, organizan y estructuran las necesidades identificadas durante la etapa de levantamiento o elicitación de requisitos. Su propósito es comprender con claridad qué debe hacer el sistema, cómo debe comportarse y qué restricciones o condiciones deben considerarse durante su desarrollo.

En esta etapa se transforman las necesidades iniciales de los usuarios y de la organización en requisitos claros, coherentes y bien definidos. Para ello, el equipo de trabajo revisa la información obtenida, identifica posibles inconsistencias, elimina ambigüedades y organiza los requisitos de manera lógica, facilitando su comprensión y posterior implementación.

El análisis de requisitos también permite identificar relaciones entre las diferentes funcionalidades del sistema, determinar dependencias entre requisitos y establecer prioridades de desarrollo. Este proceso contribuye a que el equipo de desarrollo tenga una visión estructurada del sistema que se desea construir.

Además, el análisis de requisitos favorece la comunicación entre los diferentes actores del proyecto, como usuarios, analistas, desarrolladores y responsables del negocio, ya que proporciona una base común para discutir, ajustar y validar las necesidades del sistema antes de iniciar su construcción.

Para llevar a cabo esta actividad de manera efectiva, la ingeniería de software emplea diversas técnicas y herramientas que permiten organizar, evaluar y estructurar la información obtenida. Entre ellas se destacan la priorización de requisitos, las matrices de trazabilidad y la descomposición funcional, las cuales facilitan la

comprensión del alcance del sistema y apoyan la toma de decisiones durante el desarrollo del proyecto.

1.1. Técnicas de análisis de requisitos

Las técnicas de análisis de requisitos permiten examinar, organizar y comprender la información recopilada durante la etapa de elicitación, con el fin de transformar las necesidades de los usuarios en requisitos claros, estructurados y comprensibles para el equipo de desarrollo. Estas técnicas facilitan la identificación de relaciones entre los requisitos, la detección de inconsistencias y la definición del alcance funcional del sistema.

Durante el análisis de requisitos, el equipo de trabajo revisa la información obtenida mediante entrevistas, cuestionarios, talleres o revisión documental, y la estructura de forma que pueda ser utilizada posteriormente en la especificación y diseño del sistema. Este proceso contribuye a reducir ambigüedades y a garantizar que todos los actores del proyecto tengan una comprensión compartida de las funcionalidades esperadas.

Entre las principales técnicas utilizadas en el análisis de requisitos se encuentran:

a) Identificación y clasificación de requisitos

Consiste en revisar la información obtenida y organizar los requisitos según su tipo, por ejemplo, funcionales, no funcionales o de restricción. Esta clasificación facilita su comprensión y permite establecer una estructura clara para su documentación.

b) Análisis de dependencias entre requisitos

Permite identificar relaciones entre los diferentes requisitos del sistema. Algunos requisitos pueden depender de otros para poder implementarse, por lo que comprender estas relaciones ayuda a planificar adecuadamente el desarrollo del software.

c) Agrupación por funcionalidades o módulos

Esta técnica consiste en organizar los requisitos según las áreas funcionales del sistema, por ejemplo, gestión de usuarios, procesamiento de información, generación de reportes o administración del sistema. De esta manera se facilita la comprensión de cómo se estructura el sistema.

d) Modelado de procesos o funcionalidades

Se utilizan representaciones gráficas o diagramas para visualizar cómo interactúan los diferentes componentes del sistema o cómo se ejecutan ciertos procesos. Estas representaciones ayudan a comprender mejor el funcionamiento esperado del sistema.

e) Análisis de viabilidad técnica y organizacional

En esta técnica se evalúa si los requisitos identificados pueden desarrollarse con los recursos disponibles, considerando aspectos técnicos, económicos y organizacionales. Este análisis permite ajustar los requisitos antes de iniciar el desarrollo del sistema.

La aplicación de estas técnicas permite obtener una visión estructurada del sistema que se desea construir, facilitando la toma de decisiones durante el desarrollo del proyecto y contribuyendo a que los requisitos estén correctamente definidos antes de su especificación formal.

1.2. Priorización de requisitos

La priorización de requisitos corresponde al proceso mediante el cual se establece el orden de importancia de los requisitos identificados para el sistema. Este proceso permite determinar cuáles funcionalidades deben desarrollarse primero y cuáles pueden implementarse en etapas posteriores del proyecto.

En el desarrollo de software, no todos los requisitos tienen el mismo nivel de importancia. Algunos son esenciales para el funcionamiento del sistema, mientras que otros representan mejoras o funcionalidades adicionales. Por esta razón, la priorización facilita la toma de decisiones cuando existen limitaciones de tiempo, presupuesto o recursos técnicos.

La priorización de requisitos se realiza generalmente con la participación de diferentes actores del proyecto, como usuarios, clientes, analistas y equipo de desarrollo, quienes evalúan la importancia de cada requisito según su impacto en el negocio, el valor que aporta al usuario y la viabilidad de implementación.

Entre los criterios que suelen utilizarse para priorizar requisitos se encuentran:

- **Valor para el negocio:** determina qué tan importante es el requisito para cumplir los objetivos del sistema o de la organización.
- **Dependencias entre requisitos:** algunos requisitos dependen de otros para poder implementarse, por lo que su prioridad puede verse influenciada por estas relaciones.
- **Impacto en el usuario:** identifica qué funcionalidades aportan mayor beneficio o utilidad para los usuarios finales.
- **Complejidad de implementación:** analiza el nivel de esfuerzo técnico requerido para desarrollar el requisito.
- **Riesgo del proyecto:** ciertos requisitos pueden implicar mayores riesgos técnicos o de integración, por lo que pueden abordarse en etapas tempranas para reducir incertidumbres.

Una de las técnicas más utilizadas para priorizar requisitos consiste en clasificarlos según su nivel de importancia.

Tabla 1. Ejemplo de clasificación de requisitos según prioridad

Nivel de prioridad	Descripción
Alta	Requisitos indispensables para el funcionamiento del sistema. Deben implementarse en las primeras etapas del desarrollo.
Media	Requisitos importantes que aportan valor al sistema, pero que pueden desarrollarse después de los requisitos críticos.

Nivel de prioridad	Descripción
Baja	Requisitos opcionales o mejoras que pueden incorporarse en futuras versiones del sistema.

La correcta priorización de requisitos permite planificar de manera más eficiente el desarrollo del software, optimizar el uso de los recursos disponibles y asegurar que las funcionalidades más importantes del sistema se implementen en las primeras etapas del proyecto.

1.3. Matrices de trazabilidad

Las matrices de trazabilidad son herramientas utilizadas en la gestión de requisitos que permiten establecer relaciones entre los diferentes elementos del proyecto de software. Su propósito principal es garantizar que todos los requisitos definidos sean considerados durante las distintas etapas del desarrollo y que cada uno de ellos esté correctamente implementado, probado y validado.

La trazabilidad facilita el seguimiento de los requisitos desde su origen hasta su implementación final en el sistema. De esta manera, es posible identificar cómo cada requisito se relaciona con actividades de diseño, desarrollo, pruebas y validación. Esto permite verificar que ningún requisito sea omitido y que todas las funcionalidades del sistema correspondan a necesidades previamente definidas.

Además, la matriz de trazabilidad resulta útil para gestionar cambios en los requisitos. Cuando se modifica o se elimina un requisito, la matriz permite identificar rápidamente qué componentes del sistema podrían verse afectados, lo que contribuye a reducir errores y facilitar la toma de decisiones durante el desarrollo.

Entre los principales beneficios de utilizar matrices de trazabilidad se encuentran:

- Permiten verificar que todos los requisitos estén implementados en el sistema.
- Facilitan el seguimiento de los requisitos a lo largo del ciclo de vida del software.
- Apoyan el control de cambios en los requisitos del proyecto.
- Mejoran la organización y documentación del desarrollo del software.
- Contribuyen a asegurar la calidad del sistema desarrollado.

Una matriz de trazabilidad suele organizarse en forma de tabla, donde se relacionan los requisitos con diferentes elementos del proyecto.

Tabla 2. Ejemplo de matriz de trazabilidad de requisitos

ID del requisito	Descripción del requisito	Módulo del sistema	Caso de prueba	Estado
RQ-01	El usuario debe poder registrarse en la plataforma.	Módulo de usuarios.	CP-01 Registro de usuario.	Implementado.

ID del requisito	Descripción del requisito	Módulo del sistema	Caso de prueba	Estado
RQ-02	El sistema debe permitir iniciar sesión.	Módulo de autenticación.	CP-02 Inicio de sesión.	En desarrollo.
RQ-03	El usuario puede recuperar su contraseña.	Módulo de seguridad.	CP-03 Recuperación de contraseña.	Pendiente.
RQ-04	El sistema debe generar reportes de uso.	Módulo de reportes.	CP-04 Generación de reportes.	Implementado.

La matriz de trazabilidad constituye un instrumento fundamental para asegurar la coherencia entre los requisitos del sistema y las actividades realizadas durante el desarrollo del software. Su utilización permite mantener un control claro sobre el avance del proyecto y facilita los procesos de verificación y validación del sistema.

1.4. Descomposición funcional

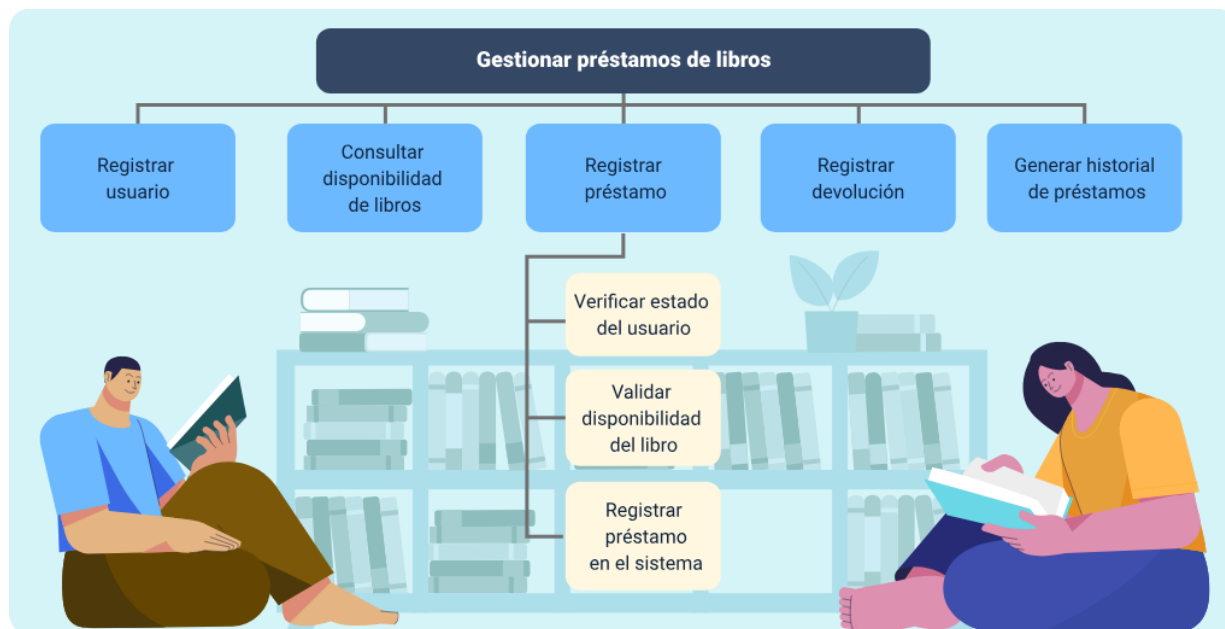
La descomposición funcional es una técnica de análisis de requisitos que consiste en dividir un sistema o proceso complejo en funciones más pequeñas y manejables. Este enfoque permite comprender de manera organizada las actividades que debe realizar el sistema, facilitando la identificación de requisitos, la asignación de responsabilidades y la estructuración lógica de las funcionalidades.

A través de la descomposición funcional se parte de una función general del sistema y se subdivide progresivamente en subfunciones o tareas específicas. Este proceso ayuda a clarificar el alcance del sistema, identificar dependencias entre procesos y facilitar el diseño posterior del software.

En el análisis de requisitos, esta técnica permite representar de forma estructurada las operaciones que el sistema debe ejecutar, lo que contribuye a mejorar la comunicación entre analistas, desarrolladores y demás interesados en el proyecto.

Ejemplo de descomposición funcional: en un sistema de gestión de biblioteca, la función principal puede ser gestionar préstamos de libros. Esta función se divide en varias subfunciones que permiten cumplir con el proceso completo.

Figura 1. Descomposición funcional



Descomposición funcional

- Gestionar préstamos de libros
- Registrar usuario
 - Verificar estado del usuario
 - Validar disponibilidad del libro
 - Registrar préstamo en el sistema
- Consultar disponibilidad de libros
- Registrar préstamo
- Registrar devolución
- Generar historial de préstamos

En este esquema se presenta como una función principal se divide en funciones más específicas, lo que facilita comprender la estructura del sistema y definir con mayor precisión los requisitos funcionales.

2. Especificación de requisitos de software

La especificación de requisitos de software es el proceso mediante el cual se documentan de manera clara, estructurada y detallada las funcionalidades, restricciones y características que debe cumplir un sistema. Su propósito es establecer una descripción precisa de lo que se espera del software, permitiendo que todos los involucrados en el proyecto compartan una comprensión común de los objetivos y alcances del sistema.

Durante esta etapa, los requisitos identificados en el proceso de análisis se organizan y formalizan en documentos que sirven como guía para el diseño, desarrollo, pruebas y mantenimiento del software. Una buena especificación contribuye a reducir ambigüedades, evitar interpretaciones incorrectas y facilitar la comunicación entre analistas, desarrolladores, clientes y demás actores del proyecto.

Además, la especificación de requisitos permite establecer criterios claros para evaluar si el sistema desarrollado cumple con las necesidades planteadas inicialmente. Por esta razón, se utilizan diferentes estándares, plantillas y técnicas de documentación que garantizan la calidad, coherencia y trazabilidad de los requisitos a lo largo del ciclo de vida del software.

2.1. Estándares internacionales para la especificación de requisitos

Los estándares internacionales para la especificación de requisitos establecen lineamientos y buenas prácticas que orientan la forma en que deben documentarse los requisitos de un sistema de software. Su objetivo es garantizar que la información sea

clara, completa, consistente y comprensible para todos los actores involucrados en el desarrollo del proyecto.

Estos estándares proporcionan estructuras y criterios que facilitan la organización de los requisitos dentro de documentos formales, evitando ambigüedades y mejorando la comunicación entre analistas, desarrolladores, clientes y usuarios. Asimismo, permiten asegurar la calidad de la documentación y facilitar actividades posteriores como el diseño, la implementación, las pruebas y el mantenimiento del sistema.

Uno de los estándares más utilizados es IEEE 830, el cual define una estructura para la elaboración de la Especificación de Requisitos del Software (ERS). Este estándar propone incluir secciones como la descripción general del sistema, los requisitos funcionales, los requisitos no funcionales, las interfaces del sistema y las restricciones del proyecto.

Posteriormente, el estándar ISO/IEC/IEEE 29148 amplía y actualiza estas recomendaciones, proporcionando directrices más completas para la ingeniería de requisitos en el desarrollo de software y sistemas. Este estándar promueve una documentación organizada que facilite la trazabilidad, validación y gestión de los requisitos durante todo el ciclo de vida del software.

El uso de estos estándares permite que los equipos de desarrollo trabajen con documentos estructurados, comparables y fácilmente interpretables, lo que contribuye a mejorar la calidad del software y a reducir errores durante el proceso de desarrollo.

2.2. Plantillas ERS (Especificación de Requisitos del Software)

Las plantillas de Especificación de Requisitos del Software (ERS), son formatos estructurados utilizados para documentar de manera organizada todos los requisitos de un sistema de software. Su propósito es establecer una forma clara y uniforme de describir las necesidades del sistema, facilitando la comunicación entre analistas, desarrolladores, clientes y demás participantes del proyecto.

Estas plantillas permiten registrar información relevante sobre el sistema, garantizando que los requisitos sean comprensibles, verificables y trazables a lo largo del ciclo de vida del desarrollo de software. Además, contribuyen a evitar ambigüedades y a mejorar la calidad de la documentación técnica.

Existen diferentes tipos de plantillas ERS que se utilizan según el enfoque metodológico o el estándar adoptado en el proyecto. Algunas de las más utilizadas son:

- Plantilla basada en el estándar IEEE 830

Ampliamente utilizada en proyectos tradicionales de desarrollo de software.

- Plantilla basada en el estándar ISO/IEC/IEEE 29148

Que actualiza y amplía los lineamientos para la ingeniería de requisitos.

- Plantillas simplificadas para metodologías ágiles

Que documentan requisitos mediante historias de usuario y criterios de aceptación.

En general, una plantilla ERS suele incluir las siguientes secciones principales:

- a) Introducción
 - Propósito del documento
 - Alcance del sistema
 - Definiciones y abreviaturas
- b) Descripción general del sistema
 - Perspectiva del producto
 - Funciones del sistema
 - Características de los usuarios
- c) Requisitos específicos
 - Requisitos funcionales
 - Requisitos no funcionales
 - Requisitos de interfaz
- d) Restricciones del sistema
- e) Criterios de aceptación y validación

El uso de estas plantillas permite estandarizar la documentación de requisitos, facilitar su análisis y asegurar que todos los participantes del proyecto tengan una comprensión común de las funcionalidades y características que debe cumplir el sistema.

De esta manera, las plantillas ERS se convierten en un instrumento clave dentro del proceso de ingeniería de requisitos, ya que apoyan la organización de la información, fortalecen la comunicación entre los actores del proyecto y sirven como base para las fases posteriores de diseño, desarrollo, pruebas y validación del software.

2.3. Técnicas de documentación de requisitos en proyectos tradicionales

En los proyectos de desarrollo de software basados en enfoques tradicionales, la documentación de requisitos se caracteriza por ser formal, estructurada y detallada. Generalmente, los requisitos se definen en las primeras etapas del proyecto y se registran en documentos técnicos que sirven como referencia durante todo el ciclo de desarrollo.

Este enfoque busca describir con precisión las funcionalidades del sistema, las restricciones, los requisitos no funcionales y las condiciones que debe cumplir el software. De esta manera, se facilita la planificación del proyecto, se reducen ambigüedades y se asegura que todos los participantes tengan una comprensión clara de lo que se debe desarrollar.

Entre las principales técnicas utilizadas para documentar requisitos en proyectos tradicionales se encuentran:

a) Documento de Especificación de Requisitos del software (ERS)

Es el documento formal donde se registran de manera estructurada todos los requisitos del sistema, incluyendo requisitos funcionales, no funcionales, interfaces, restricciones y criterios de aceptación. Este documento sirve como base para el diseño, desarrollo y pruebas del sistema.

b) Diagramas de casos de uso

Permiten representar gráficamente las interacciones entre los usuarios (actores) y el sistema. Estos diagramas facilitan la comprensión de las funcionalidades que debe ofrecer el software y ayudan a identificar los diferentes escenarios de uso.

c) Diagramas UML (Lenguaje Unificado de Modelado)

Incluyen diferentes tipos de diagramas, como diagramas de clases, secuencia, actividad o componentes. Estas representaciones permiten modelar la estructura y el comportamiento del sistema de manera visual.

d) Modelos funcionales del sistema

Describen cómo se procesan los datos dentro del sistema y cómo se relacionan sus diferentes funciones. Pueden representarse mediante diagramas de flujo, diagramas de procesos o modelos de descomposición funcional.

e) Matrices de trazabilidad de requisitos

Permiten relacionar los requisitos con otros elementos del proyecto, como casos de uso, módulos del sistema, pruebas o entregables. Esto facilita el seguimiento y la verificación de que todos los requisitos han sido implementados.

La documentación detallada en este tipo de proyectos resulta especialmente útil en sistemas complejos, en proyectos de gran escala o en aquellos donde los requisitos deben mantenerse relativamente estables durante el desarrollo del software.

2.4. Técnicas de documentación de requisitos en proyectos ágiles

En los proyectos de desarrollo de software basados en metodologías ágiles, la documentación de requisitos se caracteriza por ser más flexible, ligera y adaptable a los cambios. A diferencia de los enfoques tradicionales, en los que se busca documentar todos los requisitos desde el inicio del proyecto, en los enfoques ágiles los requisitos se refinan y ajustan de manera progresiva durante el desarrollo.

Este tipo de documentación prioriza la comunicación constante entre los miembros del equipo y los usuarios del sistema, permitiendo incorporar mejoras o modificaciones conforme se obtienen nuevos aprendizajes sobre las necesidades del producto.

Entre las principales técnicas utilizadas para documentar requisitos en proyectos ágiles se encuentran:

a) Historias de usuario

Son descripciones breves y sencillas de una funcionalidad del sistema desde la perspectiva del usuario. Suelen redactarse con una estructura simple que describe quién necesita la funcionalidad, qué necesita y con qué propósito, facilitando la comprensión del valor que aporta cada requisito.

b) Criterios de aceptación

Son condiciones específicas que deben cumplirse para considerar que una historia de usuario ha sido implementada correctamente. Estos criterios permiten

validar que la funcionalidad desarrollada responde a las necesidades definidas por el usuario.

c) Product backlog

Es una lista priorizada de todos los requisitos, funcionalidades, mejoras o correcciones que se desean incorporar en el sistema. Este listado se actualiza de manera continua a lo largo del proyecto y sirve como guía para la planificación del trabajo del equipo de desarrollo.

d) Sprint backlog

Contiene el conjunto de historias de usuario o tareas que el equipo se compromete a desarrollar durante un ciclo de trabajo corto denominado sprint. Este backlog se define al inicio de cada iteración.

e) Tableros de seguimiento del trabajo

Son herramientas visuales que permiten organizar y monitorear el avance de las tareas del equipo. Generalmente se utilizan tableros tipo Kanban donde se visualizan estados como pendiente, en desarrollo y finalizado.

Estas técnicas permiten mantener una documentación clara y suficiente para el desarrollo del sistema, sin generar una carga excesiva de documentación. De esta forma, se favorece la adaptabilidad del proyecto y la entrega continua de valor al usuario.

3. Validación de requisitos de software

La validación de requisitos de software es el proceso mediante el cual se verifica que los requisitos definidos para un sistema reflejen correctamente las necesidades y expectativas de los usuarios y demás interesados del proyecto. Su propósito es asegurar que el sistema que se desarrollará responda de manera adecuada a los objetivos del negocio y a los problemas que se desean resolver.

Este proceso permite identificar inconsistencias, ambigüedades, omisiones o errores en la documentación de requisitos antes de iniciar o continuar con las etapas de diseño y desarrollo del software. De esta manera, se reducen riesgos, retrabajos y costos asociados a modificaciones tardías dentro del ciclo de vida del proyecto.

La validación de requisitos se realiza mediante diferentes criterios y técnicas que facilitan el análisis de la información documentada y permiten confirmar que los requisitos sean claros, completos, consistentes y verificables. Para ello, es común involucrar a diferentes participantes del proyecto, como analistas, desarrolladores, usuarios finales y responsables del negocio.

En este contexto, la validación se convierte en una actividad fundamental dentro de la ingeniería de requisitos, ya que contribuye a garantizar la calidad del software y a asegurar que el producto final cumpla con las funcionalidades y características esperadas. En los siguientes apartados se presentan los principales criterios, técnicas y mecanismos utilizados para llevar a cabo la validación de requisitos de software.

3.1. Criterios de validación de requisitos

Los criterios de validación de requisitos corresponden a un conjunto de características que permiten evaluar si los requisitos definidos para un sistema de software están correctamente formulados y cumplen con las condiciones necesarias para ser utilizados durante el proceso de desarrollo. Estos criterios facilitan verificar que los requisitos representen adecuadamente las necesidades del usuario y que puedan implementarse, probarse y mantenerse de manera efectiva.

Aplicar criterios de validación permite detectar problemas en la documentación de requisitos antes de que el sistema sea construido, lo cual reduce errores, retrabajos y costos durante las etapas posteriores del proyecto.

Entre los criterios más utilizados para validar requisitos de software se encuentran los siguientes:

- **Claridad**

Un requisito debe estar redactado de forma comprensible, evitando ambigüedades o interpretaciones múltiples. Todas las personas involucradas en el proyecto deben poder entenderlo de la misma manera.

- **Viabilidad o factibilidad**

El requisito debe poder implementarse con los recursos, tecnologías, tiempo y presupuesto disponibles para el proyecto.

- **Compleitud**

El requisito debe contener toda la información necesaria para describir la funcionalidad o característica del sistema. Esto incluye condiciones, restricciones y comportamientos esperados.

- **Verificabilidad**

El requisito debe formularse de manera que sea posible comprobar su cumplimiento mediante pruebas, revisiones o demostraciones del sistema.

- **Consistencia**

Los requisitos no deben contradecirse entre sí. Cada requisito debe ser coherente con los demás y con los objetivos generales del sistema.

- **Trazabilidad**

Cada requisito debe poder relacionarse con su origen, como una necesidad del usuario, una normativa o un objetivo del negocio. Asimismo, debe ser posible rastrear su relación con el diseño, el desarrollo y las pruebas del sistema.

- **Corrección o validez**

El requisito debe representar una necesidad real del usuario o del negocio. Para ello, se valida con los interesados del proyecto con el fin de confirmar que lo documentado corresponde a lo que realmente se necesita.

- **Prioridad**

Los requisitos deben tener un nivel de importancia o prioridad definido, lo que permite organizar su implementación y facilitar la toma de decisiones durante el desarrollo del proyecto.

La aplicación sistemática de estos criterios permite mejorar la calidad de la documentación de requisitos y asegurar que el sistema de software que se desarrolle responda de manera adecuada a las necesidades de los usuarios y de la organización.

3.2. Técnicas de validación de requisitos

La validación de requisitos comprende el conjunto de técnicas utilizadas para verificar que los requisitos definidos para un sistema de software representen correctamente las necesidades de los usuarios, sean comprensibles y puedan implementarse de manera efectiva. Estas técnicas permiten identificar inconsistencias, ambigüedades o errores en la documentación antes de iniciar el desarrollo, lo que contribuye a mejorar la calidad del sistema y a reducir riesgos durante el proyecto.

Las técnicas de validación pueden aplicarse en diferentes momentos del proceso de ingeniería de requisitos y generalmente involucran la participación de analistas, desarrolladores, usuarios y otros interesados del proyecto.

Entre las técnicas más utilizadas para la validación de requisitos se encuentran las siguientes:

- **Revisión de requisitos:** consiste en la lectura y análisis sistemático de los requisitos por parte de los miembros del equipo del proyecto y de los interesados. Durante esta revisión se identifican errores, inconsistencias o requisitos incompletos. Este proceso permite asegurar que los requisitos estén correctamente documentados y alineados con los objetivos del sistema.
- **Inspecciones formales:** son evaluaciones estructuradas realizadas por un grupo de especialistas que analizan detalladamente la documentación de requisitos. Este proceso sigue procedimientos definidos y roles específicos, como moderador, autor y revisores, con el fin de detectar defectos en los requisitos antes de avanzar a las siguientes etapas del desarrollo.
- **Entrevistas con usuarios y partes interesadas:** a través del diálogo directo con los usuarios o interesados del proyecto se verifica si los requisitos documentados reflejan adecuadamente sus necesidades y expectativas. Esta técnica permite aclarar dudas, ampliar información y validar la pertinencia de los requisitos definidos.
- **Prototipado:** consiste en la construcción de representaciones preliminares del sistema, como interfaces o modelos funcionales, que permiten a los usuarios visualizar cómo funcionará el sistema. Los prototipos facilitan la retroalimentación de los usuarios y ayudan a validar si los requisitos cumplen con sus expectativas.
- **Casos de uso y escenarios:** el análisis de casos de uso y escenarios de interacción permite comprobar si los requisitos describen correctamente el comportamiento esperado del sistema frente a distintas situaciones. Esta

técnica ayuda a identificar omisiones o inconsistencias en los requisitos funcionales.

- **Listas de verificación (checklists):** se utilizan listas de criterios previamente definidos para revisar cada requisito y verificar si cumple con características como claridad, consistencia, completitud y verificabilidad. Esta técnica facilita realizar evaluaciones sistemáticas y estandarizadas.

La aplicación combinada de estas técnicas permite mejorar la calidad de los requisitos documentados, asegurar su comprensión por parte de los diferentes actores del proyecto y garantizar que el sistema de software responda de manera adecuada a las necesidades planteadas por los usuarios y la organización.

3.3. Revisiones, auditorías y prototipos

La validación de requisitos de software también se apoya en mecanismos estructurados que permiten examinar de manera sistemática la documentación generada durante el proceso de ingeniería de requisitos. Entre estos mecanismos se destacan las revisiones, las auditorías y el uso de prototipos, los cuales contribuyen a garantizar que los requisitos sean correctos, completos y coherentes con las necesidades del sistema.

- **Revisiones de requisitos**

Las revisiones consisten en el análisis detallado de la documentación de requisitos por parte de los miembros del equipo de desarrollo y otros actores involucrados en el proyecto, como analistas, líderes técnicos o usuarios representantes. Durante este proceso se examina la claridad, consistencia, viabilidad y pertinencia de

los requisitos documentados. Las revisiones permiten detectar errores, ambigüedades o duplicidades en etapas tempranas del proyecto, lo que facilita realizar correcciones oportunas antes de iniciar el desarrollo del sistema.

- **Auditorías de requisitos**

Las auditorías corresponden a evaluaciones formales que se realizan para verificar que los requisitos definidos cumplan con estándares, metodologías y lineamientos establecidos por la organización o por modelos de calidad de software. Generalmente son realizadas por personal independiente del equipo que elaboró los requisitos, con el fin de garantizar objetividad en la evaluación. Las auditorías permiten comprobar la trazabilidad de los requisitos, su correcta documentación y su alineación con los objetivos del proyecto.

- **Prototipos**

prototipado consiste en la elaboración de representaciones preliminares del sistema que permiten visualizar su funcionamiento antes de su implementación definitiva. Estos prototipos pueden presentarse como bocetos de interfaces, modelos interactivos o simulaciones parciales del sistema. Su principal objetivo es facilitar la interacción con los usuarios para validar si los requisitos definidos responden a sus expectativas y necesidades reales. A través de la retroalimentación obtenida durante el uso del prototipo, es posible ajustar o redefinir requisitos que no resulten adecuados o que requieran mayor precisión.

El uso de revisiones, auditorías y prototipos fortalece el proceso de validación de requisitos, ya que permite evaluar la calidad de la información documentada desde diferentes perspectivas y con distintos niveles de formalidad, contribuyendo así al

desarrollo de sistemas de software más confiables y alineados con los objetivos del proyecto.

3.4. Manual de usuario como mecanismo de validación

El manual de usuario constituye un documento fundamental dentro del proceso de validación de requisitos de software, ya que describe de manera clara y detallada cómo debe utilizarse el sistema desde la perspectiva del usuario final. Su elaboración permite verificar que las funcionalidades definidas en los requisitos se encuentren correctamente implementadas y que el sistema responda a las necesidades y expectativas de quienes lo utilizarán.

Durante la elaboración del manual de usuario se revisan las funcionalidades del sistema, los procesos que debe ejecutar el usuario y las condiciones necesarias para su correcto funcionamiento. Este proceso facilita identificar inconsistencias entre los requisitos definidos y el comportamiento real del sistema, lo que convierte al manual en una herramienta útil para validar la correcta interpretación de los requisitos.

Además, el manual de usuario contribuye a mejorar la comprensión del sistema, ya que traduce los requisitos técnicos en instrucciones claras y comprensibles para los usuarios finales, permitiendo confirmar que las funcionalidades del sistema son intuitivas, coherentes y adecuadas para su contexto de uso.

En general, un manual de usuario suele organizarse en las siguientes secciones:

a) Introducción

Presenta el propósito del manual, el público al que está dirigido y una descripción general del sistema.

- **Descripción general del sistema**

Explica las principales funcionalidades del sistema y el contexto en el que será utilizado.

- **Requisitos para el uso del sistema**

Describe las condiciones técnicas necesarias para utilizar el sistema, como hardware, software o configuraciones específicas.

- **Acceso al sistema**

Indica los procedimientos para ingresar al sistema, incluyendo autenticación o creación de usuarios.

- **Descripción de funcionalidades**

Explica paso a paso las operaciones que puede realizar el usuario dentro del sistema, generalmente acompañadas de instrucciones claras y ordenadas.

- **Procedimientos de uso**

Presenta guías o secuencias de acciones que el usuario debe seguir para realizar tareas específicas dentro del sistema.

- **Manejo de errores o problemas comunes**

Describe posibles errores o dificultades que pueden presentarse y las recomendaciones para solucionarlos.

- **Glosario o términos técnicos**

Incluye definiciones de términos utilizados en el sistema para facilitar la comprensión del usuario.

De esta manera, el manual de usuario no solo funciona como una guía para la utilización del sistema, sino también como un mecanismo que permite verificar si las funcionalidades desarrolladas cumplen con los requisitos definidos durante la etapa de análisis.

4. Gestión de requisitos en proyectos de software

La gestión de requisitos en proyectos de software corresponde al conjunto de procesos, técnicas y actividades orientadas a identificar, organizar, controlar y mantener actualizados los requisitos del sistema durante todo el ciclo de vida del desarrollo. Su propósito principal es asegurar que las necesidades del cliente y de los usuarios finales se comprendan correctamente, se documenten de manera adecuada y se mantengan alineadas con los objetivos del proyecto.

En los proyectos de desarrollo de software, los requisitos no son elementos estáticos. A lo largo del proceso pueden surgir cambios derivados de nuevas necesidades del negocio, ajustes tecnológicos, restricciones del entorno o mejoras identificadas durante las etapas de diseño, desarrollo y pruebas. Por esta razón, la gestión de requisitos permite controlar dichos cambios de forma organizada, evitando inconsistencias y garantizando que todas las modificaciones sean evaluadas, aprobadas y documentadas.

Una adecuada gestión de requisitos también facilita la trazabilidad, es decir, la posibilidad de rastrear cada requisito desde su origen hasta su implementación, pruebas y validación. Esto permite verificar que todas las funcionalidades definidas en la etapa de análisis se encuentren correctamente desarrolladas y que el sistema cumpla con los objetivos planteados inicialmente.

Dentro de la gestión de requisitos se desarrollan diferentes actividades fundamentales que permiten mantener la coherencia y calidad de la información del proyecto. Entre las más importantes se encuentran:

- **Identificación de requisitos:** consiste en reconocer y registrar las necesidades, expectativas y restricciones del sistema a partir de la interacción con clientes, usuarios y demás interesados del proyecto.
- **Documentación de requisitos:** implica describir de manera estructurada los requisitos funcionales y no funcionales utilizando formatos o plantillas que faciliten su comprensión y análisis.
- **Priorización de requisitos:** permite establecer el nivel de importancia de cada requisito, teniendo en cuenta factores como el valor para el negocio, el impacto en el sistema o la complejidad de implementación.
- **Control de cambios:** se refiere al proceso mediante el cual se gestionan las modificaciones a los requisitos, evaluando su impacto en el alcance, el tiempo y los recursos del proyecto.
- **Trazabilidad de requisitos:** permite relacionar cada requisito con los elementos del sistema que lo implementan, así como con las pruebas que verifican su cumplimiento.
- **Seguimiento y monitoreo:** consiste en revisar continuamente el estado de los requisitos para asegurar que se desarrollen conforme a lo planificado y que se mantenga la coherencia entre los diferentes componentes del proyecto.

Una gestión adecuada de requisitos contribuye a reducir errores en el desarrollo, mejorar la comunicación entre los participantes del proyecto y garantizar que el software entregado responda de manera efectiva a las necesidades para las cuales fue diseñado. En consecuencia, este proceso se convierte en un elemento clave para el

éxito de los proyectos de software, especialmente en entornos donde los cambios y la evolución de los sistemas son frecuentes.

4.1. Contratos de software: requisitos fijos y variables

En los proyectos de desarrollo de software, los contratos establecen los acuerdos formales entre el cliente y el proveedor respecto a las características, funcionalidades, plazos, costos y responsabilidades asociadas al sistema que se desarrollará. Dentro de estos contratos, los requisitos del software pueden clasificarse en requisitos fijos y requisitos variables, dependiendo del grado de flexibilidad permitido durante el desarrollo del proyecto.

Los requisitos fijos corresponden a aquellas funcionalidades o características del sistema que han sido definidas desde el inicio del proyecto y que deben cumplirse obligatoriamente según lo establecido en el contrato. Estos requisitos no pueden modificarse fácilmente durante el proceso de desarrollo, ya que representan compromisos formales entre las partes. Generalmente se utilizan en proyectos con enfoques tradicionales de desarrollo, donde el alcance del sistema se define de manera detallada desde las etapas iniciales.

Por otro lado, los requisitos variables son aquellos que pueden ajustarse o modificarse durante el desarrollo del proyecto en función de nuevas necesidades, cambios en el entorno organizacional o retroalimentación de los usuarios. Este tipo de requisitos se gestiona mediante procedimientos de control de cambios previamente definidos en el contrato. Los requisitos variables son más comunes en proyectos que

utilizan metodologías ágiles, donde se promueve la adaptación continua a las necesidades del cliente.

Ejemplo práctico: en el desarrollo de un sistema de gestión de reservas para un hotel, el contrato de software puede establecer diferentes tipos de requisitos.

Requisitos fijos:

- El sistema debe permitir registrar reservas de habitaciones.
- El sistema debe almacenar la información de los clientes.
- El sistema debe generar confirmaciones de reserva.
- El sistema debe garantizar la seguridad y protección de los datos de los usuarios.

Estos requisitos se consideran obligatorios y forman parte del alcance principal del proyecto.

Requisitos variables:

- Integración con pasarelas de pago adicionales.
- Implementación de un módulo de reportes avanzados.
- Integración con aplicaciones móviles o sistemas externos.
- Incorporación de funcionalidades adicionales solicitadas por el cliente durante el desarrollo.

Estos requisitos pueden modificarse, ampliarse o ajustarse durante el proyecto mediante acuerdos entre el cliente y el equipo de desarrollo.

La diferenciación entre requisitos fijos y variables permite establecer con mayor claridad el alcance del proyecto, definir mecanismos de control frente a cambios y evitar conflictos entre las partes involucradas. Asimismo, contribuye a mejorar la planificación, el seguimiento del desarrollo del software y la gestión de riesgos asociados a modificaciones en los requisitos.

En la práctica, muchos contratos de software combinan ambos tipos de requisitos, estableciendo un conjunto de funcionalidades obligatorias que deben cumplirse y un conjunto de requisitos susceptibles de cambio, los cuales pueden ajustarse mediante acuerdos formales durante el desarrollo del proyecto. De esta manera se busca equilibrar la estabilidad del proyecto con la flexibilidad necesaria para adaptarse a nuevas necesidades.

Tabla 3. Comparación entre requisitos fijos y requisitos variables

Característica	Requisitos fijos	Requisitos variables
Definición	Son requisitos definidos desde el inicio del proyecto y que deben cumplirse obligatoriamente según lo establecido en el contrato.	Son requisitos que pueden modificarse o ajustarse durante el desarrollo del proyecto.

Característica	Requisitos fijos	Requisitos variables
Flexibilidad	Presentan baja flexibilidad, ya que los cambios suelen implicar modificaciones contractuales.	Presentan mayor flexibilidad y pueden adaptarse a nuevas necesidades del cliente o del entorno.
Momento de definición	Se establecen en las etapas iniciales del proyecto.	Pueden definirse o modificarse durante las iteraciones del desarrollo.
Metodologías asociadas	Comunes en metodologías tradicionales o en modelos predictivos de desarrollo.	Frecuentes en metodologías ágiles que permiten adaptación continua.
Gestión de cambios	Requieren procesos formales de gestión contractual para cualquier modificación.	Se gestionan mediante mecanismos de control de cambios dentro del proyecto.

4.2. Herramientas para la gestión de requisitos

Las herramientas para la gestión de requisitos son aplicaciones o plataformas que permiten registrar, organizar, analizar, priorizar y hacer seguimiento a los requisitos de un sistema de software durante todo el ciclo de vida del proyecto. Su uso facilita la administración de la información, la comunicación entre los miembros del equipo de desarrollo y el control de los cambios que puedan surgir en los requisitos.

Estas herramientas ayudan a centralizar la documentación de los requisitos, permitiendo que analistas, desarrolladores, gestores de proyecto y clientes tengan acceso a una fuente de información común. De esta manera se reduce el riesgo de inconsistencias, pérdida de información o interpretaciones erróneas durante el proceso de desarrollo.

Entre las principales funciones que ofrecen las herramientas de gestión de requisitos se encuentran:

- Registro y documentación estructurada de requisitos funcionales y no funcionales.
- Priorización y organización de requisitos según su importancia o impacto en el proyecto.
- Control de versiones y seguimiento de cambios en los requisitos.
- Establecimiento de relaciones de trazabilidad entre requisitos, casos de uso, diseño, desarrollo y pruebas.
- Colaboración entre los miembros del equipo mediante comentarios, revisiones y actualizaciones compartidas.
- Generación de reportes para facilitar el seguimiento y la toma de decisiones en el proyecto.

El uso de estas herramientas resulta especialmente importante en proyectos de software de mediana y gran escala, donde la cantidad de requisitos y la participación de múltiples actores hacen necesario contar con mecanismos que permitan mantener la información organizada y actualizada.

En la actualidad existen diversas herramientas utilizadas en la industria del software para apoyar la gestión de requisitos. Algunas de las más conocidas son:

IBM Engineering Requirements Management DOORS: herramienta ampliamente utilizada en proyectos complejos de ingeniería y desarrollo de sistemas. Permite gestionar grandes volúmenes de requisitos y mantener trazabilidad detallada entre ellos.

- **Jira Software**: plataforma utilizada principalmente en metodologías ágiles para gestionar historias de usuario, tareas y seguimiento del desarrollo del software.
- **Azure DevOps**: herramienta que permite gestionar requisitos, historias de usuario, tareas, pruebas y control del desarrollo dentro de un entorno integrado.
- **ReqView**: herramienta especializada en gestión y trazabilidad de requisitos que facilita la organización y análisis de la documentación del proyecto.
- **Confluence**: plataforma colaborativa utilizada para documentar requisitos, especificaciones y conocimientos del proyecto, especialmente en entornos ágiles.

El uso adecuado de estas herramientas contribuye a mejorar la calidad de los requisitos, facilitar la trazabilidad de la información y garantizar que el sistema desarrollado cumpla con las necesidades y expectativas de los usuarios y del cliente.

4.3. Tipos y características de herramientas de gestión de requisitos

Las herramientas de gestión de requisitos pueden clasificarse en diferentes tipos según las funcionalidades que ofrecen, el tipo de proyecto en el que se utilizan y el enfoque metodológico que apoyan. Estas herramientas permiten organizar, documentar, analizar y dar seguimiento a los requisitos del sistema, facilitando la coordinación entre los distintos participantes del proyecto.

En términos generales, las herramientas de gestión de requisitos se pueden agrupar en los siguientes tipos:

- a) Herramientas especializadas en gestión de requisitos:** son herramientas diseñadas específicamente para la ingeniería de requisitos. Permiten registrar requisitos de forma estructurada, establecer relaciones de trazabilidad, gestionar versiones y controlar cambios en los requisitos del sistema. Estas herramientas suelen utilizarse en proyectos complejos o en organizaciones que desarrollan sistemas de gran escala.

Ejemplos de este tipo de herramientas incluyen IBM Engineering Requirements Management DOORS y ReqView, que permiten administrar grandes volúmenes de requisitos y mantener un seguimiento detallado de su evolución durante el desarrollo del proyecto.

- b) Herramientas de gestión de proyectos con soporte para requisitos:** son plataformas que, además de gestionar tareas, cronogramas y actividades del proyecto, incorporan funcionalidades para registrar y organizar requisitos, historias de usuario o funcionalidades del sistema. Estas

herramientas son ampliamente utilizadas en equipos de desarrollo que trabajan con metodologías ágiles.

Entre las más utilizadas se encuentran Jira Software y Azure DevOps, que permiten administrar historias de usuario, backlog de producto, incidencias y seguimiento del desarrollo del software.

c) Herramientas colaborativas de documentación: son plataformas que permiten crear, organizar y compartir documentación del proyecto de manera colaborativa. Aunque no están diseñadas exclusivamente para la gestión de requisitos, suelen utilizarse para documentar especificaciones, historias de usuario, manuales y otros artefactos relacionados con el desarrollo del software.

Un ejemplo ampliamente utilizado es Confluence, que permite a los equipos documentar requisitos, mantener registros del proyecto y facilitar la comunicación entre los participantes.

Además de su clasificación por tipo, las herramientas de gestión de requisitos presentan una serie de características comunes que facilitan su uso dentro de los proyectos de desarrollo de software.

Tabla 4. Características de las herramientas de gestión de requisitos

Característica	Descripción
Gestión estructurada de requisitos	Permite registrar, organizar y clasificar los requisitos funcionales y no funcionales del sistema de forma ordenada.

Característica	Descripción
Trazabilidad	Posibilita relacionar los requisitos con otros elementos del proyecto como casos de uso, diseño, desarrollo y pruebas.
Control de versiones	Permite mantener un historial de cambios realizados en los requisitos a lo largo del proyecto.
Gestión de cambios	Facilita registrar, evaluar y aprobar modificaciones en los requisitos del sistema.
Colaboración entre equipos	Permite que diferentes miembros del proyecto participen en la revisión, actualización y validación de los requisitos.
Generación de reportes	Apoya el seguimiento del avance del proyecto mediante informes que permiten analizar el estado de los requisitos.

El uso de estas herramientas contribuye a mejorar la organización de la información del proyecto, fortalecer la comunicación entre los participantes y garantizar que los requisitos del sistema se gestionen de manera adecuada durante todo el ciclo de vida del desarrollo del software.

4.4. Control y gestión de cambios en los requisitos

Durante el desarrollo de un proyecto de software es común que los requisitos evolucionen debido a nuevas necesidades del cliente, cambios en el entorno organizacional, avances tecnológicos o retroalimentación obtenida durante el proceso de desarrollo. Por esta razón, resulta fundamental implementar mecanismos de control y gestión de cambios en los requisitos, con el fin de asegurar que cualquier modificación sea analizada, aprobada y documentada de manera adecuada.

La gestión de cambios en los requisitos consiste en un conjunto de procedimientos y prácticas que permiten registrar, evaluar, aprobar e implementar modificaciones en los requisitos del sistema sin afectar la estabilidad del proyecto. Este proceso busca mantener la coherencia entre los requisitos definidos, el diseño del sistema, el desarrollo del software y las pruebas realizadas.

Un proceso adecuado de control de cambios contribuye a evitar inconsistencias en la documentación, prevenir errores en el desarrollo del sistema y garantizar que todos los participantes del proyecto tengan claridad sobre las modificaciones realizadas.

De manera general, el proceso de gestión de cambios en los requisitos suele incluir las siguientes etapas:

a) Solicitud de cambio

Un miembro del equipo, un usuario o el cliente identifica la necesidad de modificar, eliminar o agregar un requisito. Esta solicitud se registra formalmente mediante un documento o herramienta de gestión de requisitos.

b) Análisis del impacto del cambio

El equipo del proyecto analiza cómo afectará la modificación a otros requisitos, al diseño del sistema, al cronograma, a los costos y a los recursos del proyecto.

c) Evaluación y aprobación del cambio

El cambio es evaluado por los responsables del proyecto, quienes determinan si la modificación es viable y si debe ser aprobada, rechazada o pospuesta.

d) Implementación del cambio

Una vez aprobado, el cambio se incorpora en la documentación de requisitos y se realizan los ajustes necesarios en el diseño, desarrollo y pruebas del sistema.

e) Actualización y seguimiento

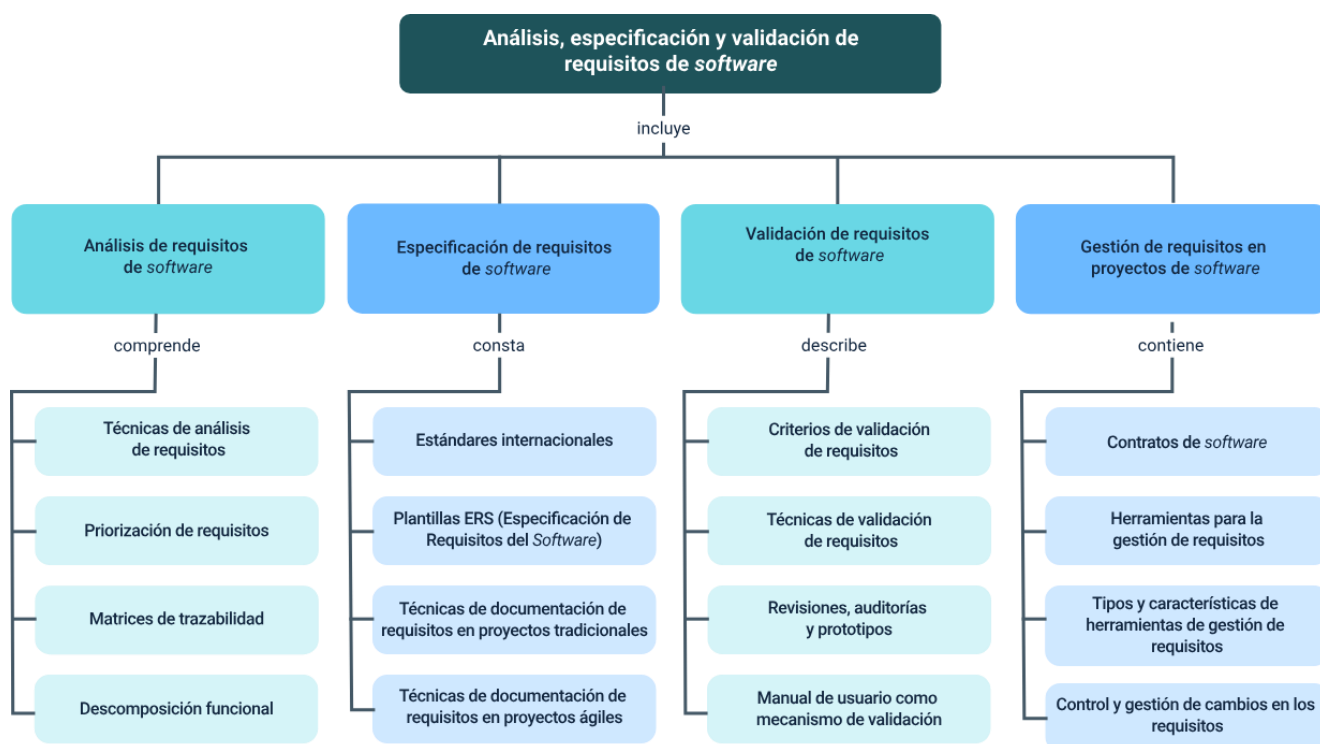
Se actualizan los documentos y herramientas de gestión de requisitos para mantener la trazabilidad del cambio y asegurar que todos los participantes conozcan la modificación realizada.

En muchos proyectos de software, especialmente en entornos ágiles, los cambios en los requisitos se gestionan de manera continua mediante reuniones de planificación, revisiones de producto y retroalimentación constante con los usuarios.

Implementar un adecuado control de cambios permite mantener la coherencia del proyecto, reducir riesgos asociados a modificaciones no controladas y garantizar que el software final responda de manera efectiva a las necesidades reales de los usuarios.

Síntesis

El componente formativo contiene el análisis, la especificación, la validación y la gestión de requisitos en proyectos de desarrollo de software. Se estudian técnicas para analizar y priorizar requisitos, el uso de matrices de trazabilidad y la descomposición funcional. Además, se presentan estándares internacionales y plantillas ERS para documentar requisitos en enfoques tradicionales y ágiles. Finalmente, se analizan los criterios y técnicas de validación, el uso de revisiones, auditorías, prototipos y manuales de usuario, así como herramientas y estrategias para la gestión y control de cambios en los requisitos durante el ciclo de vida del software.



Glosario

Análisis de requisitos: proceso mediante el cual se identifican, examinan y estructuran las necesidades del sistema que se desarrollará. Permite comprender las expectativas de los usuarios y transformarlas en requisitos claros para el desarrollo del software.

Auditoría de requisitos: proceso de revisión formal que permite verificar si los requisitos del sistema cumplen con criterios de calidad, coherencia, completitud y alineación con los objetivos del proyecto.

Control de cambios: procedimiento mediante el cual se registran, analizan, aprueban o rechazan las modificaciones realizadas a los requisitos del sistema durante el desarrollo del proyecto.

Descomposición funcional: técnica de análisis que consiste en dividir un sistema o proceso complejo en funciones más pequeñas y manejables, con el fin de facilitar su comprensión y diseño.

Herramientas de gestión de requisitos: aplicaciones o plataformas que permiten registrar, organizar, rastrear y administrar los requisitos de un sistema durante todo el ciclo de vida del desarrollo del software.

Matriz de trazabilidad: instrumento que permite relacionar los requisitos del software con otros elementos del proyecto, como el diseño, la implementación y las pruebas, con el fin de garantizar su seguimiento durante el desarrollo.

Prototipo: representación preliminar de un sistema o de una parte de él que permite visualizar funcionalidades y validar requisitos antes del desarrollo definitivo del software.

Trazabilidad de requisitos: capacidad de seguir el origen, evolución y relación de los requisitos a lo largo de todas las etapas del desarrollo del software.

Validación de requisitos: proceso mediante el cual se verifica que los requisitos definidos reflejan correctamente las necesidades del usuario y que son completos, consistentes y comprensibles.

Versionamiento de requisitos: mecanismo que permite mantener un historial de cambios realizados en los requisitos del sistema, facilitando su seguimiento y control durante el proyecto.

Referencias bibliográficas

IEEE 830

ISO/IEC/IEEE 29148

IEEE Computer Society. (2014). Guía del SWEBOK: guía para el cuerpo de conocimiento de la ingeniería de software (3.ª ed.). IEEE Computer Society Press.

International Organization for Standardization, International Electrotechnical Commission, & IEEE. (2018). ISO/IEC/IEEE 29148:2018. Ingeniería de sistemas y software - Procesos del ciclo de vida - Ingeniería de requisitos. ISO.

Kendall, Kenneth E., & Kendall, Julie E. (2011). Análisis y diseño de sistemas (8.ª ed.). Pearson Educación.

Larman, Craig (2003). UML y patrones: introducción al análisis y diseño orientado a objetos (2.ª ed.). Prentice Hall.

Organización Internacional de Normalización. (2018). ISO/IEC/IEEE 29148: Ingeniería de sistemas y software. Procesos del ciclo de vida. Requisitos. ISO.

Pressman, Roger S., & Maxim, Bruce R. (2020). Ingeniería del software: un enfoque práctico (9.ª ed.). McGraw-Hill Education.

Sommerville, Ian (2016). Ingeniería del software (10.ª ed.). Pearson Educación.

Sommerville, Ian, & Sawyer, Pete (2011). Ingeniería de requisitos: procesos y técnicas. Pearson Educación.

Créditos

Nombre	Cargo	Centro de Formación y Regional
Claudia Johanna Gómez Pérez	Responsable Ecosistema de Recursos Educativos Digitales (RED)	Centro Agroturístico - Regional Santander
Diana Rocío Possos Beltrán	Responsable de línea de producción	Centro de Comercio y Servicios - Regional Tolima
Viviana Esperanza Herrera Quiñonez	Evaluada instrucción	Centro de Comercio y Servicios - Regional Tolima
Jose Yobani Penagos Mora	Diseñador de contenidos digitales	Centro de Comercio y Servicios - Regional Tolima
Sebastian Trujillo Afanador	Desarrollador full stack	Centro de Comercio y Servicios - Regional Tolima
Gilberto Junior Rodríguez Rodríguez	Animador y productor audiovisual	Centro de Comercio y Servicios - Regional Tolima
Jorge Eduardo Rueda Peña	Evaluaor de contenidos inclusivos y accesibles	Centro de Comercio y Servicios - Regional Tolima

Nombre	Cargo	Centro de Formación y Regional
Jorge Bustos Gómez	Validador y vinculator de recursos educativos digitales	Centro de Comercio y Servicios - Regional Tolima